

## Day 9: Autoregressive Language Models

---

- **Autoregressive Language Models** — Build & train a decoder-only LM
- 3 hours lecture + practical
- Slides, notes, and code on the course site

- Back to Autoregressive (Week 2 Arc)
- From Text to Tokens
- The Decoder-Only Transformer Block
- Training One Model End-to-End

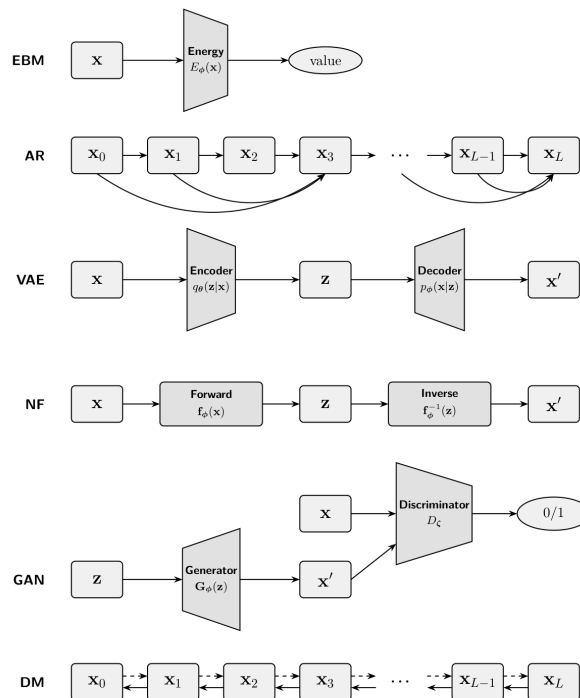
## 1 · Back to Autoregressive (Week 2 Arc)

---

## 1.1 Where We Are: the Generative Map (Day 6)

- Day 6 goal: transport noise to the data distribution
- Likelihood **exact**: autoregressive, normalizing flows
- Likelihood **bound**: VAEs, diffusion; **implicit**: GANs
- Days 6-8 went deep on diffusion / score / flow
- Today we return to the **autoregressive** family — LLMs

# 1.1 Where We Are: the Generative Map (Day 6)



**Figure 1.2: Computation graphs of prominent deep generative models.** Top to bottom: **EBM** maps an input  $x$  to a scalar energy; **AR** generates a sequence  $\{x_t\}_{t=0}^L$  left to right with causal dependencies; **VAE** encodes  $x$  to a latent  $z$  and decodes to a reconstruction  $x'$ ; **NF** applies an invertible map  $f_\phi$  between  $x$  and  $z$  and uses  $f_\phi^{-1}$  to produce  $x'$ ; **GAN** transforms noise  $z$  to a sample  $x'$  that is judged against real  $x$  by a discriminator  $D_G$ ; **DM** iteratively refines a noisy sample through a multi-step denoising chain  $\{x_t\}_{t=0}^L$ . Boxes denote variables, trapezoids are learnable networks, ovals are scalars; arrows indicate computation flow. The trapezoid was intended to indicate a dimension-changing transformation, while rectangles denote variables or mappings that preserve dimensionality.

## 1.2 Autoregressive vs Diffusion

- AR:  $p(x) = \prod_t p(x_t \mid x_{<t})$  — exact likelihood
- Diffusion: ELBO / score matching — bound or implicit
- AR training: next-token cross-entropy
- AR sampling: **one token at a time** (Day 10: KV cache)
- Diffusion sampling: **many steps** along a schedule / ODE

## 1.3 The Autoregressive Factorization

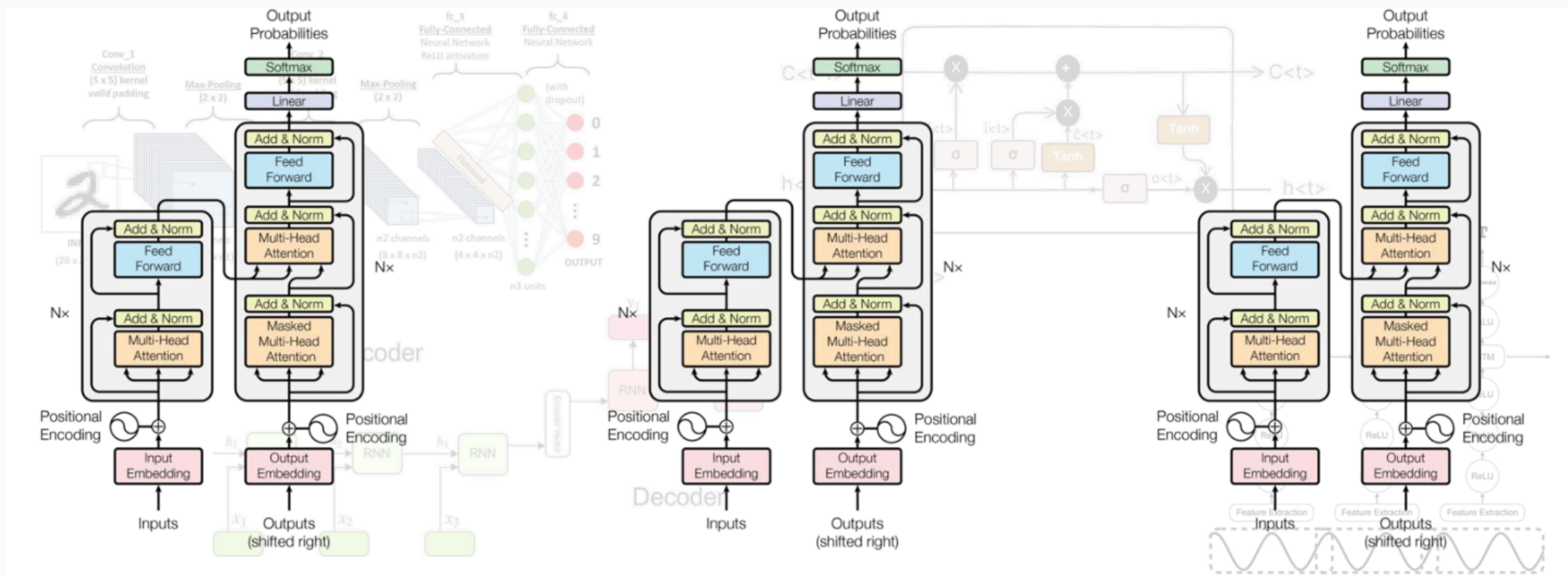
- Chain rule:  $p(x) = \prod_{t=1}^T p(x_t \mid x_{<t})$
- Model every conditional with one shared network
- Train by maximum likelihood = next-token cross-entropy
- Causal mask (Day 5) computes all conditionals in one pass
- Generation = ancestral sampling, token by token



## 1.4 Architecture Families

- Encoder-only (BERT): bidirectional, for understanding
- Encoder-decoder (T5, NMT): map one sequence to another
- Decoder-only (GPT, chat): causal, next-token prediction
- Today: the **decoder-only** stack that powers modern LLMs
- One architecture, scaled with data + compute

# 1.4 Architecture Families



## 2 · From Text to Tokens



## 2.1 Tokenization (BPE)

- Text is split into **subword** tokens, not words or characters
- Byte-pair encoding: merge frequent byte pairs greedily
- Balances vocabulary size vs sequence length
- Handles any string (rare words, code, emoji) — no OOV
- Vocabulary  $V$  30k-130k for modern LMs

## 2.2 Special Tokens & Document Packing

- Special tokens: BOS / EOS / PAD mark boundaries
- Concatenate documents into one long token stream
- Slice into fixed-length blocks (the context window)
- Packing keeps GPUs busy — no wasted padding
- Attention/loss masks stop attending across document joins

## 2.3 Embeddings

- Each token id indexes a row of an embedding table  $E \in \mathbb{R}^{V \times d}$
- Token id  $\rightarrow$  vector  $x_t \in \mathbb{R}^d$
- Often **tie** the embedding with the output `lm_head` weights
- Position is added separately (Day 5) — or via RoPE (later)
- Output of this stage: a  $T \times d$  matrix of token vectors

### 3 · The Decoder-Only Transformer Block

---

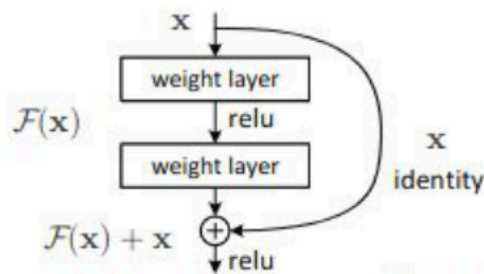
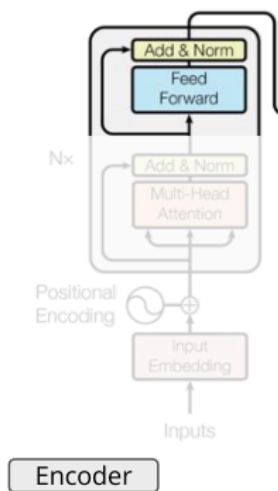
## 3.1 The Pre-Norm Residual Block

- Modern blocks normalize **before** each sublayer (pre-norm)
- $h' = h + \text{MHA}(\text{LN}(h))$
- $h'' = h' + \text{MLP}(\text{LN}(h'))$
- Residual path keeps gradients healthy through depth
- Same identity-path idea as ResNet (Day 4)



## 3.1 The Pre-Norm Residual Block

Add: A residual connection (skip connection): Helps gradients flow through deep networks → combats vanishing gradients



### Deep Residual Learning for Image Recognition

Kaiming He   Xiangyu Zhang   Shaoqing Ren   Jian Sun  
Microsoft Research  
{kahe, v-xiangz, v-shren, jiansun}@microsoft.com

Norm: A Layer Normalization which keeps output scale consistent, → speeds up and stabilizes training [Pre- Vs Post- Norm]

## 3.2 RMSNorm

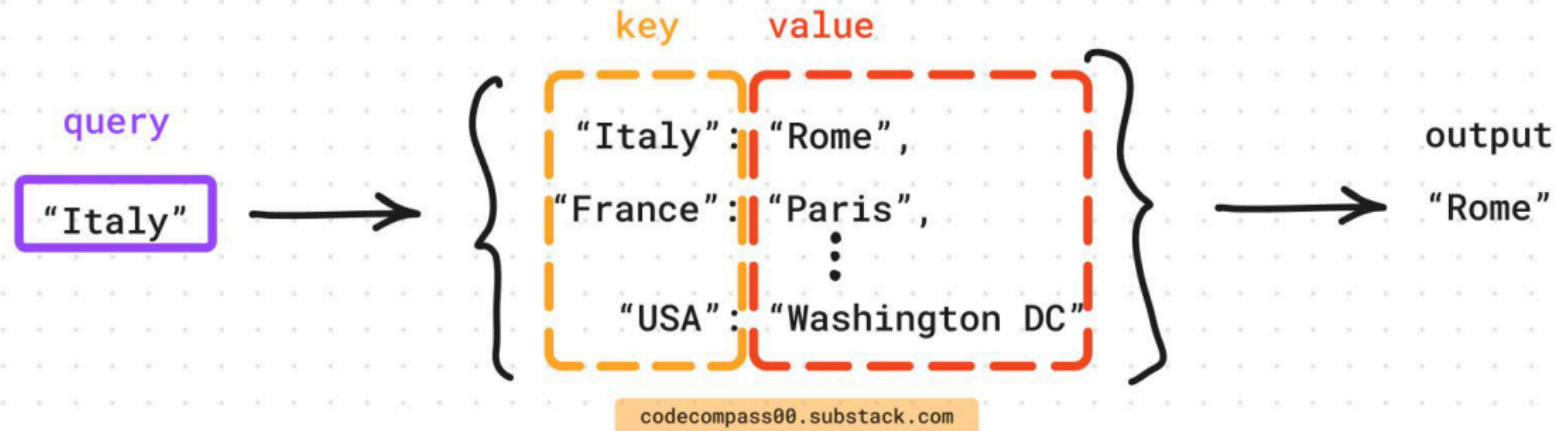
- Normalize each token vector by its root-mean-square
- $\text{RMSNorm}(x) = x \frac{1}{\text{rms}(x)} \cdot \gamma$
- No mean subtraction, no bias — cheaper than LayerNorm
- Stabilizes activations in deep stacks
- Default norm in LLaMA-style models

## 3.3 Causal Self-Attention

- Project tokens to queries, keys, values:  $Q, K, V$
- $A = \text{softmax}\left(QK^T \frac{\backslash}{\sqrt{d_k}}\right)$  – soft dictionary lookup
- Causal mask: set  $A_{i,j} = 0$  for  $j > i$  (no peeking ahead)
- Output =  $AV$  – a weighted average of past values
- This is the conditional  $p(x_t \mid x_{<t})$  in action

### 3.3 Causal Self-Attention

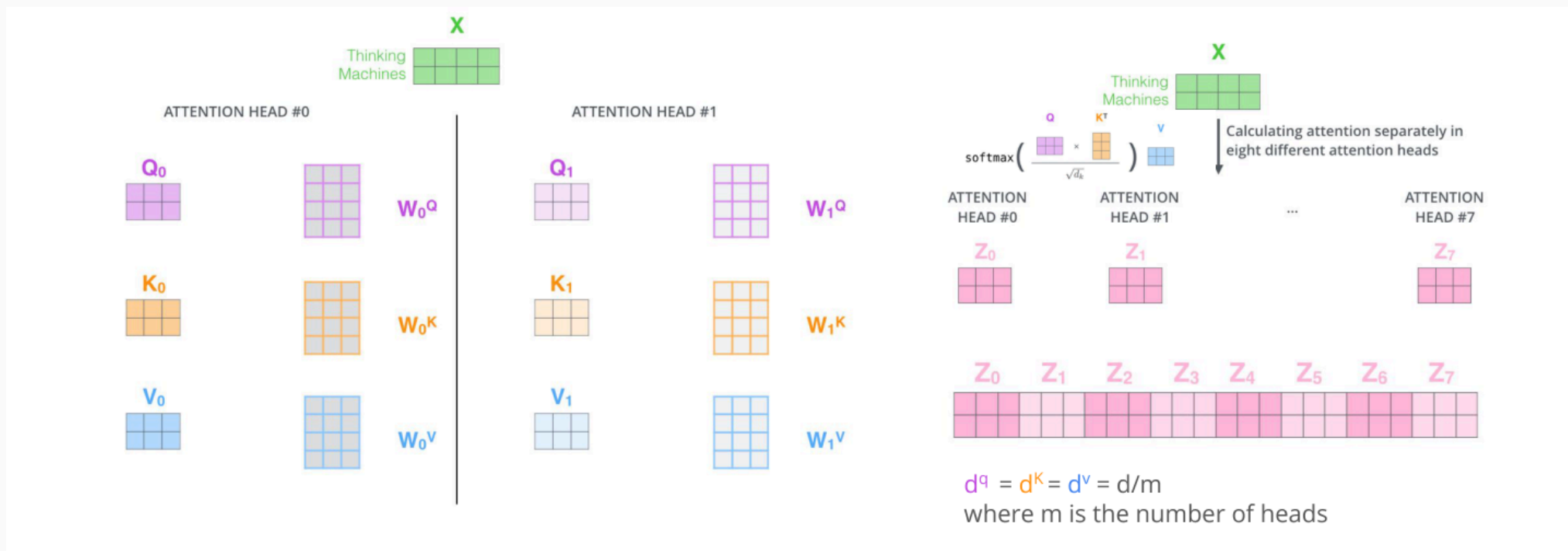
#### Traditional Dictionary



## 3.4 Multi-Head & Grouped-Query Attention

- Run  $H$  attention heads in parallel, then concatenate
- Each head learns a different relation (syntax, coref, ...)
- Multi-query / grouped-query: heads **share** K, V
- GQA shrinks the KV cache (Day 10) with little quality loss
- Heads = parallel relevance maps, like CNN filters

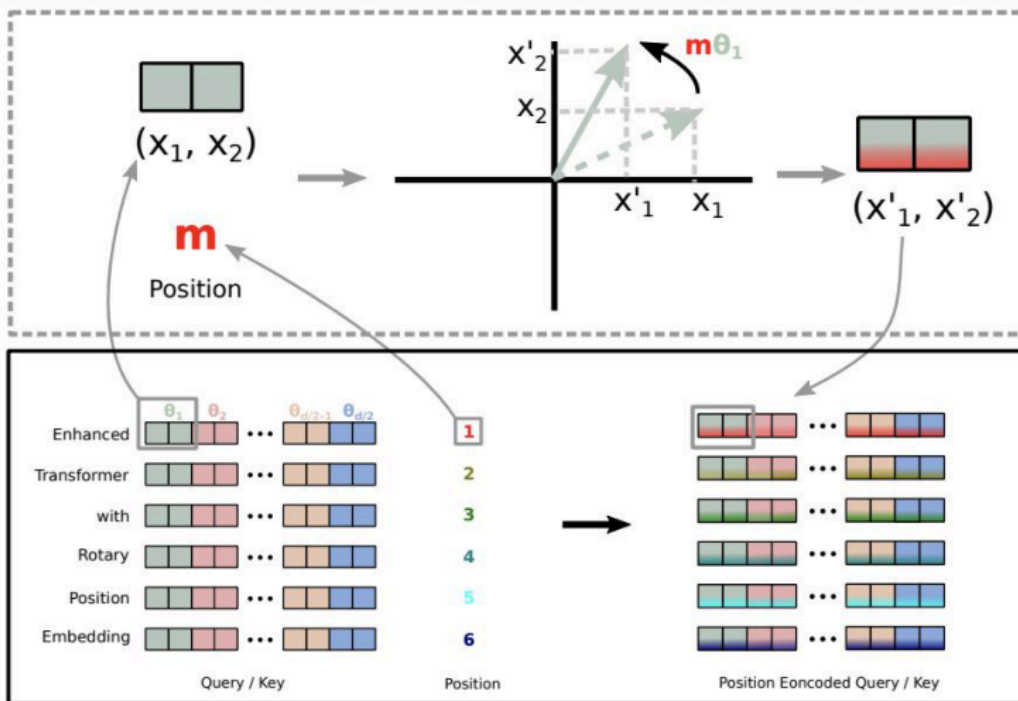
### 3.4 Multi-Head & Grouped-Query Attention



## 3.5 Rotary Position Embeddings (RoPE)

- Rotate  $Q, K$  by an angle that depends on position
- Dot product then depends on **relative** offset  $m - n$
- No learned position table; extrapolates to longer context
- YaRN: rescale frequencies to extend context length
- Standard in LLaMA, Qwen, Mistral, ...

### 3.5 Rotary Position Embeddings (RoPE)



Uses both absolute and relative information

$$f_q(\mathbf{x}_m, m) = (\mathbf{W}_q \mathbf{x}_m) e^{im\theta}$$

$$f_k(\mathbf{x}_n, n) = (\mathbf{W}_k \mathbf{x}_n) e^{in\theta}$$

$$g(\mathbf{x}_m, \mathbf{x}_n, m - n) = \text{Re}[(\mathbf{W}_q \mathbf{x}_m)(\mathbf{W}_k \mathbf{x}_n)^* e^{i(m-n)\theta}]$$

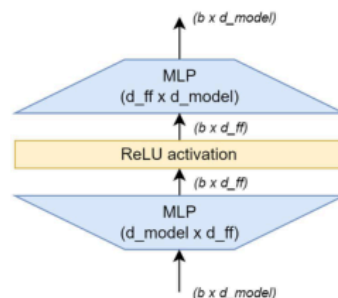
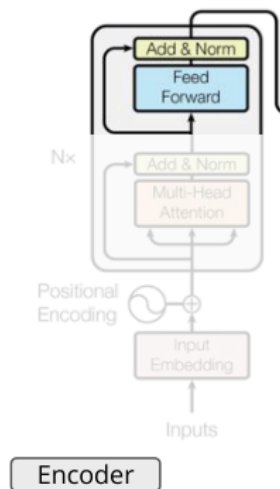


## 3.6 The MLP: GeGLU / SwiGLU

- Position-wise FFN refines each token independently
- Vanilla:  $W_2 \text{ GELU}(W_1 x)$
- Gated (GeGLU/SwiGLU):  $W_2 (\text{GELU}(W_0 x) \cdot (W_1 x))$
- Where most parameters and FLOPs live
- Hidden width typically  $4d$  (or  $\frac{8}{3}d$  for gated)

## 3.6 The MLP: GeGLU / SwiGLU

This is where most of the work happens..



$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

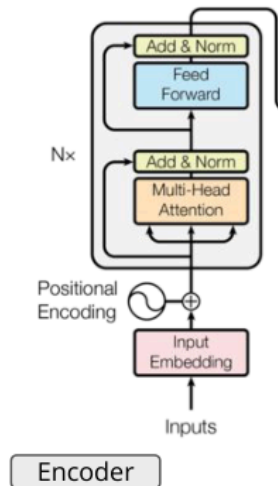
$$d_{ff} = 4 \times d$$

## 3.7 Stacking the Block to Logits

- Stack  $L$  identical blocks:  $h^{(l)} = \text{Block}(h^{(l-1)})$
- Final norm, then  $\text{lm\_head}: h \rightarrow \mathbb{R}^V$  logits
- Softmax over the vocabulary  $\rightarrow$  next-token distribution
- Same weights at every position (parameter sharing)
- This is a full GPT forward pass

## 3.7 Stacking the Block to Logits

produces representation of  $x$  that captures the meaning in context.



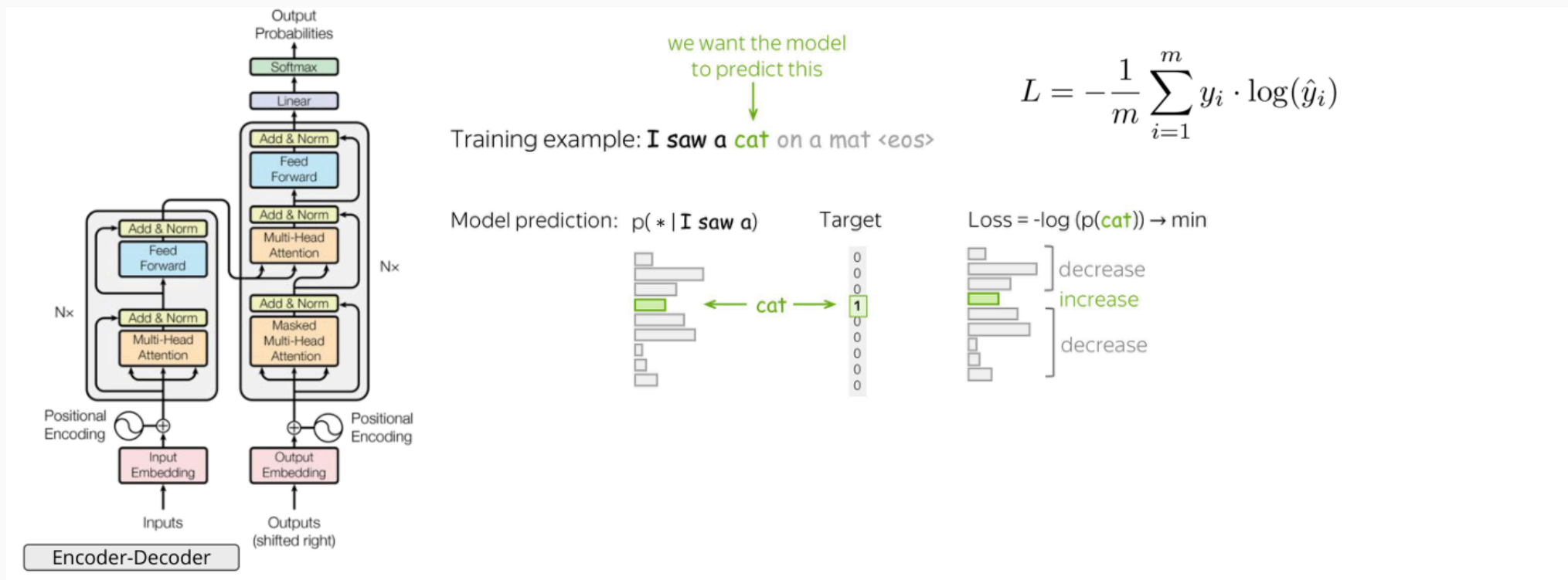
## 4 · Training One Model End-to-End

---

## next-token cross-entropy over the vocabulary

- Loss:  $L = - \sum_i \log p_\theta(x_i \mid x_{<i})$
- **next-token cross-entropy over the vocabulary**
- Teacher forcing: feed ground-truth context, predict next
- Labels = inputs shifted by one position
- Report **perplexity** =  $\exp(L)$

## 4.1 The Training Objective



## 4.2 Optimization Stack

- AdamW with weight decay
- Learning-rate warmup, then cosine decay
- Gradient clipping for stability
- Mixed precision (bf16); gradient accumulation for big batches
- Long runs over billions of tokens



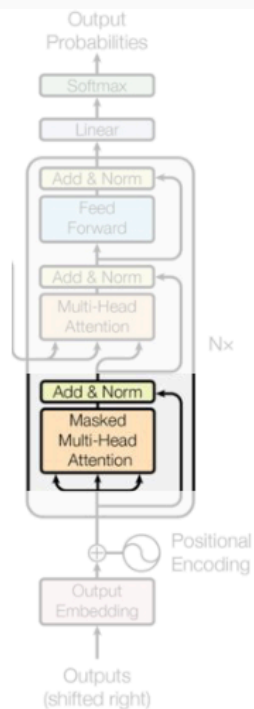
## 4.3 Parameters & FLOPs

- Params per block  $12d^2$  (attention  $4d^2$  + MLP  $8d^2$ )
- Total  $12Ld^2$  plus  $Vd$  embeddings
- Training compute  $6N$  FLOPs per token ( $N$  = params)
- Forward  $2N$ , backward  $4N$
- Order-of-magnitude reasoning guides model/data sizing

## 4.4 nanoGPT Code Map

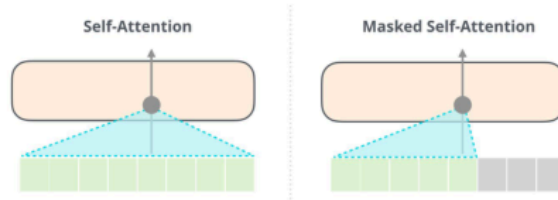
- `model.py`: GPT, Block, CausalSelfAttention, MLP
- `train.py`: data loader, training loop, optimizer
- `config`: `n_layer`, `n_head`, `n_embd`, `block_size`
- `sample.py`: generation (preview of Day 10)
- 300 lines: the whole stack you can read in an evening

## 4.4 nanoGPT Code Map



generates tokens step-by-step, predicting the next token probability  $p(y_i | x, \mathbf{y}_{<i})$  based on:

- **Previously generated tokens**



$$\text{Softmax} \left[ \frac{\text{Query Matrix} \times \text{Transposed Key Matrix}}{\sqrt{d}} \right] = \begin{bmatrix} \text{Unnormalized Attention Scores} & \rightarrow & \text{Masked Attention Scores} \end{bmatrix}$$

| Unnormalized Attention Scores |      |      |      |      | Masked Attention Scores |      |      |      |     |
|-------------------------------|------|------|------|------|-------------------------|------|------|------|-----|
| 1.1                           | 2.3  | 5.3  | 2.1  | -1.2 | 1.1                     | -∞   | -∞   | -∞   | -∞  |
| -0.1                          | 0.5  | 1.3  | -0.1 | 6.1  | -0.1                    | 0.5  | -∞   | -∞   | -∞  |
| 2.3                           | 0.2  | 3.3  | -1.0 | -0.4 | 2.3                     | 0.2  | 3.3  | -∞   | -∞  |
| 0.3                           | 1.2  | -0.3 | 5.0  | 1.6  | 0.3                     | 1.2  | -0.3 | 5.0  | -∞  |
| 5.1                           | -2.9 | -1.1 | -4.2 | 0.4  | 5.1                     | -2.9 | -1.1 | -4.2 | 0.4 |

Decoder

## 4.5 Scaling and What Is Next

- Scaling laws: loss falls predictably with params/data/compute
- Chinchilla: balance model size against training tokens
- Distributed training (data / tensor / pipeline parallel)
- Day 10: how a trained base LM is **served** and **aligned**
- Same ‘make generation fast’ theme as Day 8 — different bottleneck

- Day 9: **Autoregressive Language Models**
- Build & train a decoder-only LM
- Questions welcome — practical follows

## Questions?

Practical session follows — see course site.